Homework on Strings, Lists, Dictionaries, Tuples, and User-Defined Functions.

25 January 2026 23:48

1. Student Performance Analyzer

Write a Python program to manage student academic records.

Requirements:

- Store student data in a dictionary where the key is a student ID and the value is a tuple containing (name, class, marks_list).
- marks_list contains marks in 5 subjects.
- Create user-defined functions to:
  - Calculate the average marks of a student.
  - Assign grades based on average.
  - Display students scoring above a user-defined threshold.
- Use string formatting for neat output.

2. Word Frequency and Classification System

Create a program that analyzes a paragraph entered by the user.

Requirements:

- Remove punctuation and normalize the case using string methods.
- Store word frequencies in a dictionary.
- Categorize words into:
  - Short words (length ≤ 4)
  - Medium words (length 5–7)
  - Long words (length > 7)
- Store each category in a list.
- Write user-defined functions for:

- Frequency calculation
- Word classification
- Displaying sorted results

3. Employee Payroll Management
Design a payroll system for a company.
Requirements:

- Store employee details in a dictionary with employee ID as key and a tuple (name, department, basic_salary) as value.
- Use a list to store allowance percentages.
- Create functions to:
  - Calculate gross salary
  - Calculate tax based on salary slabs
  - Generate a formatted salary slip
- Display employees department-wise.

4. Library Book Issue Tracker
Develop a library management program.
Requirements:

- Maintain a dictionary where the key is the book ID and value is a tuple (title, author, availability_status).
- Maintain a list of issued books.
- Use functions to:
  - Issue a book
  - Return a book
  - Display available books
- Use string operations for searching books by title or author.

5. Shopping Cart and Billing System
Create an online shopping cart simulation.

Requirements:

- Store product details in a dictionary:
  product_id → (product_name, price)
- Use a list to store selected product IDs.
- Create functions to:
  - Add/remove products from cart
  - Generate a bill showing item-wise cost
  - Apply discount codes using strings
- Return the final bill as a formatted string.

6. Password Strength Evaluation System

Design a password validation tool.

Requirements:

- Accept multiple passwords in a list.
- Use string methods to check:
  - Length
  - Uppercase, lowercase, digits, special characters
- Store password and strength level (Weak, Moderate, Strong) in a dictionary.
- Use functions to:
  - Evaluate strength
  - Display summary statistics

7. Student Attendance and Report Generator

Create a program to track attendance.

Requirements:

- Store attendance data in a dictionary:
  roll_number → (student_name, attendance_list)
- attendance_list contains P or A as strings.
- Create functions to:
  - Calculate attendance percentage

- Identify defaulters (< 75%)
- Generate attendance report
- Use formatted string output.

8. Online Quiz Result Processor
Write a program to process quiz results.
Requirements:
- Store correct answers in a tuple.
- Store student responses in a dictionary:
  student_name → response_list
- Create functions to:
  - Compare answers
  - Calculate score and percentage
  - Generate rank list
- Display result summary using strings.

9. Contact Management System
Design a contact book application.
Requirements:
- Store contacts in a dictionary:
  contact_name → (phone_number, email)
- Use string methods to validate phone numbers and emails.
- Create functions to:
  - Add, update, delete contacts
  - Search contacts by name prefix
- Display contacts in alphabetical order.

10. Sales Data Analysis Tool
Create a sales analysis program.
Requirements:
- Store sales data in a dictionary:
  month → sales_list

- Each sales_list contains daily sales figures.
- Use functions to:
  - Calculate total and average sales per month
  - Identify highest and lowest sales day
  - Generate a summary report as a string
- Store monthly summaries in tuples.